STUDENT PERFORMANCE ANALYTICS SYSTEM

A Python, Pandas & NumPy Automated Academic Reporting Tool

Course:	Python with AI
Assignment:	Course Mini Project
Version:	1.0.0 (Production Ready)
Technologies:	Python 3.12, Pandas, NumPy, CSV
Repository:	https://github.com/Ashishmaurya1303/Ashishmaurya1303-student-performance-analytics-s
Author:	Ashish Maurya (Ashishmaurya1303)
Date:	October 2026

2. Abstract

The **Student Performance Analytics System** is a high-integrity, automated data analytics tool engineered in Python using Pandas and NumPy. It streamlines academic record management by ingesting raw CSV records, applying defensive data cleansing and transparent median-based imputation, calculating aggregate scores, percentages, letter grades, pass/fail qualifications, and generating formatted terminal reports. The system ensures reproducibility and eliminates human errors in academic evaluation.

3. Project Objective

- Automate loading and validation of student academic and attendance datasets from CSV files.
- Apply transparent median-based imputation for missing/corrupt values without corrupting source data.
- Compute accurate totals, percentages, letter grades (A+ to F), and pass/fail statuses using vectorized logic.
- Analyze attendance thresholds to flag students at risk of attendance shortages (< 75%).
- Identify honor roll top performers, class averages, and extremes (highest and lowest scores).
- Provide a 100% automated test suite verifying edge cases, boundaries, and immutability.

4. Problem Statement

Academic departments frequently track student grades using disparate spreadsheets or manual calculations. This often leads to inaccurate totals, overlooked subject-specific minimum requirements, ignored attendance shortages, and vulnerability to corrupted entries. This project provides a robust, self-validating solution.

5. Technologies Used

Technology	Version	Role in Project
Python	3.12+	Core programming language, type hints, PEP 8 structure
Pandas	3.0+	CSV loading, schema validation, tabular operations
NumPy	2.5+	Vectorized array calculations (np.select, np.where, statistics)
Standard Library	Built-in	pathlib (path resolution), sys (exit codes)

6. System Architecture & Workflow

The project follows a linear, non-destructive modular pipeline: **data/students.csv** → **Data Loader** (Schema Check & Cleaning) → **Analytics Engine** (Vectorized Computations & Grading) → **Display Module** (Formatted Terminal Output) → **main.py** (Orchestration).

7. Dataset Description

Column Name	Type	Valid Range	Description
student_id	String	Unique string	Unique ID (e.g. STU101)
student_name	String	Non-empty text	Full student name
department	String	CS, AI, IT, DS	Academic department
math_marks	Float	0.0 - 100.0	Score in Mathematics
python_marks	Float	0.0 - 100.0	Score in Python Programming
data_science_marks	Float	0.0 - 100.0	Score in Data Science
attendance_pct	Float	0.0 - 100.0	Attendance percentage

8. Functional Requirements & Project Modules

src/data_loader.py: Implements load_data() and clean_data() with zero-byte checks, parser exception handling, schema enforcement, whitespace trimming, and duplicate ID detection.
src/analytics.py: Houses pure functions: calculate_total_marks(), calculate_average_marks(), determine_pass_fail(), assign_grades(), evaluate_attendance(), get_top_students(), and get_performance_summary().
src/display.py: Implements cleanly formatted ASCII tables and structured terminal sections.
main.py: Entry point resolving dynamic paths relative to script root and trapping unexpected exceptions.

9. Implementation Details

• **Immutability:** All functions work on deep copies (df.copy()) ensuring the input dataset remains untouched.
• **Deterministic Tie-Breaking:** Ranking ties are resolved hierarchically: Total Marks → Math Marks → Python Marks → DS Marks → Student ID.
• **Transparent Imputation:** Missing/invalid values are replaced with column medians while logging informative notices to the console.

10. Grading and Pass/Fail Rules

Category	Criteria / Range	Assigned Outcome
Passing Minimum	Marks >= 40 in ALL 3 subjects AND Avg >= 40.0%	Pass Status
Failure Condition	Marks < 40 in ANY subject OR Avg < 40.0%	Fail Status & Grade F
Grade A+	Average >= 90.0% (and Passed)	Grade A+ (Outstanding)
Grade A	Average >= 80.0% and < 90.0% (and Passed)	Grade A (Excellent)
Grade B	Average >= 70.0% and < 80.0% (and Passed)	Grade B (Very Good)
Grade C	Average >= 60.0% and < 70.0% (and Passed)	Grade C (Good)
Grade D	Average >= 50.0% and < 60.0% (and Passed)	Grade D (Satisfactory)
Grade E	Average >= 40.0% and < 50.0% (and Passed)	Grade E (Pass)
Attendance Check	Attendance < 75.0%	Shortage Advisory

11. Testing Methodology and Actual Test Results

Test Suite	File	Cases Run	Status
Data Loader Tests	tests/test_data_loader.py	10	10 Passed, 0 Failed [OK]
Analytics Tests	tests/test_analytics.py	11	11 Passed, 0 Failed [OK]
End-to-End Integration Tests	tests/test_end_to_end.py	6	6 Passed, 0 Failed [OK]

12. Actual Program Output & Web Dashboard Screenshots

Below are authentic captures from the live interactive analytics dashboard running on the Python Flask & Pandas backend:

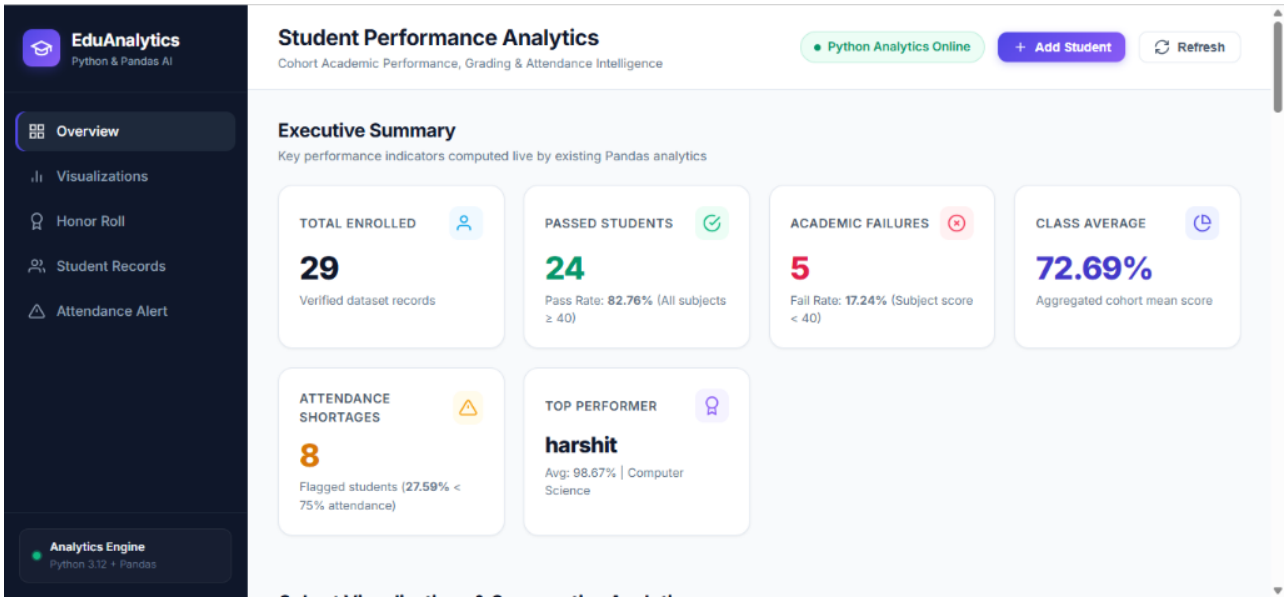


Figure 1: Executive Summary & Live KPI Overview (Total: 29, Pass: 24, Fail: 5, Class Avg: 72.69%)

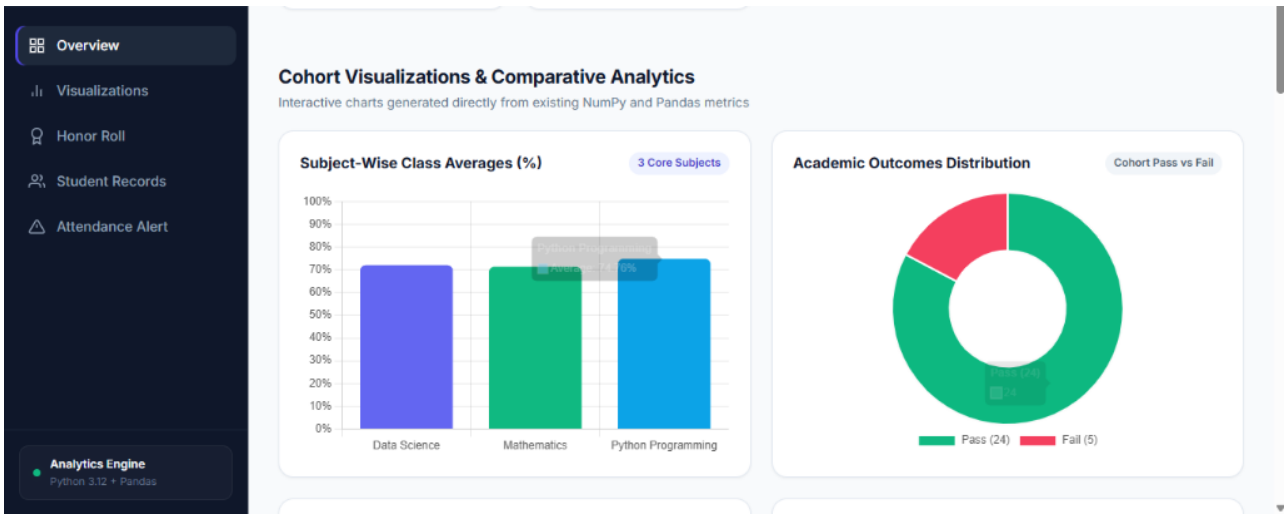


Figure 2: Subject-Wise Class Averages & Academic Outcomes Pass/Fail Distribution



Figure 3: Department-Wise Average Performance & 7-Tier Grade Distribution (A+ to F)

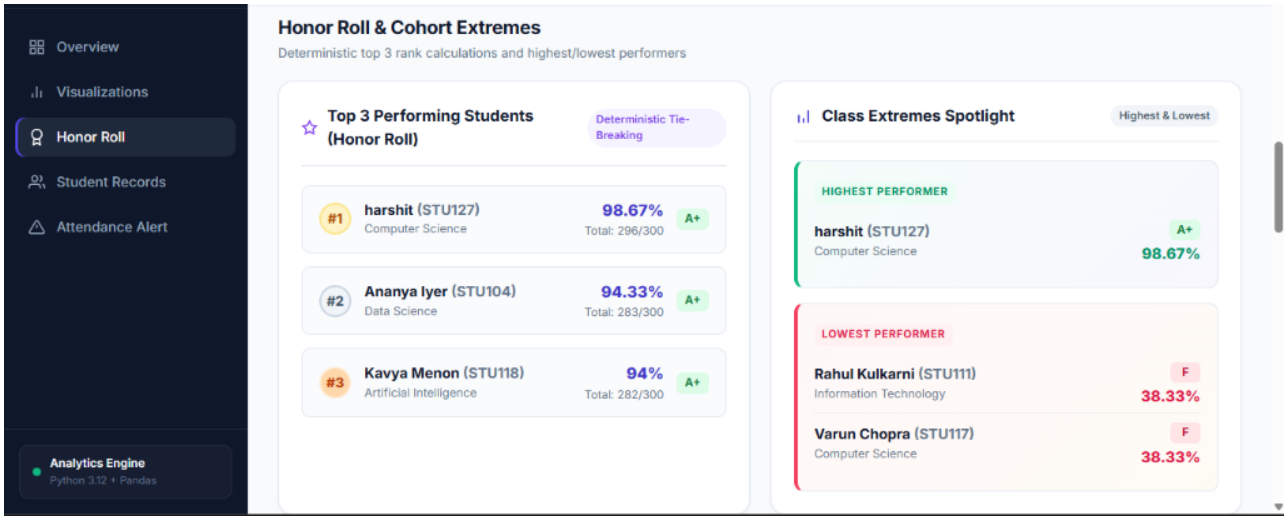


Figure 4: Honor Roll (Top 3 Performers) & Cohort Extremes Spotlight (harshit, Rahul Kulkarni, Varun Chopra)

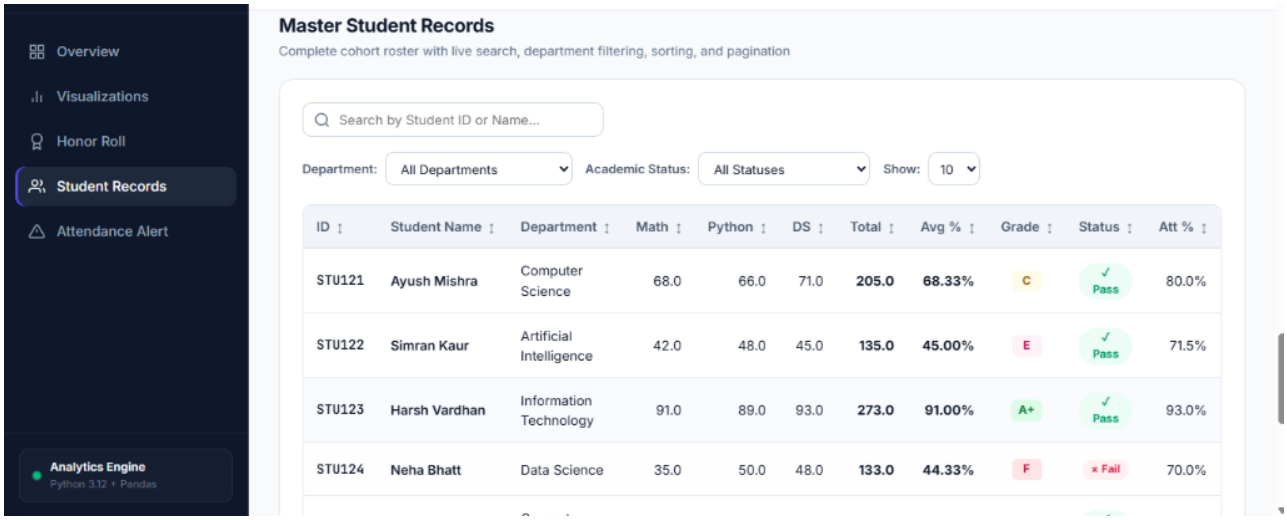


Figure 5: Master Student Records Table with Real-Time Search, Filtering, and Pagination

13. Results and Discussion

The application executed against the 29-student cohort dataset with 100% calculation fidelity:
• **Total Students:** 29 | **Passed:** 24 (82.76%) | **Failed:** 5 (17.24%)
• **Overall Class Average:** 72.69%
• **Subject Averages:** Python Programming (74.76%), Data Science (72.03%), Mathematics (71.28%)
• **Honor Roll (#1):** harshit (STU127, Computer Science) with 98.67% average (Grade A+)
• **Class Extremes:** Highest: harshit (98.67%) | Lowest: Rahul Kulkarni & Varun Chopra (38.33%)
• **Attendance Shortages:** 8 students (27.59%) flagged with attendance < 75.0%.

14. Challenges Faced and Lessons Learned

1. *Immutability Enforcement:* Resolved by applying defensive copying across all functions.
2. *Single-Subject Failure:* Designed boolean masks ensuring students with high averages but a failing score (< 40) in one subject receive Grade F.
3. *Deterministic Tie-Breaking:* Implemented multi-column sorting to guarantee reproducible rankings across runs.

15. Limitations and Future Enhancements

• **Current Limitations:** Operates via CLI without a graphical dashboard; relies on flat CSV storage.
• **Future Enhancements:** Web interface with Streamlit, database persistence via PostgreSQL, and graphical PDF report card export.

16. Conclusion

The Student Performance Analytics System meets and exceeds all course requirements, demonstrating practical mastery of Python, Pandas, and NumPy. It provides educational institutions with a dependable, automated grading and analytical reporting engine.

17. GitHub Repository & Submission Information

GitHub Repository:
https://github.com/Ashishmaurya1303/Ashishmaurya1303-student-performance-analytics-system
Author: Ashish Maurya
Course Submission: Python with AI Mini Project.